

## Day 9–10: Methods

- Method creation
- Parameters
- Return types
- Method overloading
- Recursion basics

Programs:

- Calculator using methods
- Recursive factorial
- Recursive Fibonacci

## Real Life Example First

Imagine...

Oka hotel undi.

Customer: "One Chicken Biryani."

Chef kitchen lo prepare chesi istadu.

Inko customer vachadu.

Again chef same process repeat chestadu.

Chef prati sari same steps manually explain cheyyadu.

Just order accept chesi prepare chestadu.

Programming lo kuda ade concept.

Suppose nuvvu 100 places lo addition cheyyali.

Instead of writing

a+b

100 times...

Oka method create chestav.

Whenever required

add()

call chestav.

Method ante...

**"Oka specific pani cheyyadaniki create chese reusable block of code."**

---

## **What is a Method?**

Definition:

A method is a block of code that performs a particular task whenever it is called.

Simple ga cheppali ante...

Method = Machine

Machine ni create chestam.

Whenever work kavali...

Machine start chestam.

Machine work complete chesi stop avutundi.

---

## **Example**

Without Method

```
System.out.println("Welcome");
```

```
System.out.println("Welcome");
```

```
System.out.println("Welcome");
```

```
System.out.println("Welcome");
```

Same code repeat.

Method use chesthe

```
welcome();
```

```
welcome();
```

```
welcome();
```

```
welcome();
```

Method lo code okkasari rayali.

---

## **Advantages of Methods**

Methods enduku use chestam?

### **1. Code Reuse**

Same code malli rayalsina avasaram ledi.

Example

Login system

100 pages unnayi.

Every page login check cheyyali.

Instead of

100 copies

Create

checkLogin()

Done.

---

### **2. Easy Maintenance**

Suppose login rule change ayindi.

Method use chesthe

One place change.

Whole project update.

Method lekapothe

100 places edit.

---

### **3. Readability**

Instead of

100 lines

calculateTax();

Chusthe immediate ga artham avutundi.

---

### **4. Less Bugs**

Same code copy paste chesthe mistakes ekkuva.

Method use chesthe mistakes thakkuva.

---

## Structure of Method

Syntax

```
returnType methodName(parameters)
```

```
{  
    statements;  
}
```

Example

```
int add(int a,int b)
```

```
{  
    return a+b;  
}
```

Breakdown

int

Return type

add

Method name

(int a,int b)

Parameters

return a+b;

Return value

---

## Parts of Method

Example

```
public static int add(int a,int b)
```

```
{  
    return a+b;  
}
```

Every word meaning:

## **public**

Access modifier

Means

Anywhere nundi access cheyyachu.

Ippudu deep ga avasaram ledi.

---

## **static**

Main method laage.

Object create cheyyakunda call cheyyachu.

Day OOP lo complete ga chustam.

---

## **int**

Return type

Method emi return chestundi?

Integer.

---

## **add**

Method name

Rules:

Camel Case use cheyyali.

Examples

findLargest()

calculateSalary()

printName()

isPrime()

---

## Parameters

(int a,int b)

Inputs.

---

## Body

{

...

}

Actual work.

---

## return

Final output.

---

## Method Creation

Example

public class Main

{

static void greet()

{

System.out.println("Good Morning");

}

public static void main(String args[])

{

greet();

}

}

Output

Good Morning

Flow

Program starts

↓

main()

↓

greet()

↓

prints

↓

back to main()

↓

program ends

---

## Memory Understanding

Imagine

main()

↓

Calls

↓

greet()

CPU jumps

main()

↓

greet()

↓

back

↓

main()

Method complete ayyaka control malli main ki vastundi.

---

## Method Calling

Create

```
static void hello()
```

```
{
```

```
    System.out.println("Hello");
```

```
}
```

Call

```
hello();
```

If call cheyyakapothe...

Method execute avvadu.

Method create cheyyadam okati.

Call cheyyadam inkoti.

---

## Multiple Method Calls

```
hello();
```

```
hello();
```

```
hello();
```

Output

Hello

Hello

Hello



Method body okkasari rayadam.

Execution many times.

---

## **Parameters**

Real Life

Tea machine

Without parameter

Always tea.

With parameter

Sugar

Milk

Tea powder

Different combinations.

Programming lo

Parameters = Inputs

---

Example

```
static void greet(String name)
```

```
{
```

```
    System.out.println("Hello "+name);
```

```
}
```

Call

```
greet("Ravi");
```

Output

Hello Ravi

Again

```
greet("Sita");
```

Output

Hello Sita

Same method.

Different input.

Different output.

---

### **Multiple Parameters**

```
static void add(int a,int b)
```

```
{
```

```
    System.out.println(a+b);
```

```
}
```

Call

```
add(10,20);
```

Output

30

Again

```
add(40,5);
```

Output

45

---

### **Actual Parameter**

```
add(5,7);
```

5

7

Actual values.

---

### **Formal Parameter**

```
(int a,int b)
```

Variables inside method.

---

### **Rule**

Actual count

=

Formal count

Same.

Wrong

add(10);

Needs 2 values.

---

## **Return Type**

Method work complete ayyaka value return chestundi.

Example

Calculator

Input

10

20

Output

30

---

Example

```
static int add(int a,int b)
```

```
{
```

```
    return a+b;
```

```
}
```

Call

```
int ans=add(5,7);
```

```
System.out.println(ans);
```

Output

12

---

## **void Return Type**

Suppose output ivvakapothe

Use

void

Example

```
static void hello()
```

```
{
```

```
    System.out.println("Hello");
```

```
}
```

No return.

---

## **Difference**

void

Returns nothing.

int

Returns integer.

double

Returns decimal.

boolean

Returns true/false.

String

Returns text.

char

Returns character.

---

## **Return Statement**

Example

```
return a+b;
```

Meaning

Method work complete.

Immediately back to caller.

Below return

Nothing executes.

Example

return 5;

System.out.println("Hello");

Compile Error.

Because unreachable.

---

## **Method Overloading**

One of the most important interview topics.

Definition

Same method name

Different parameters.

---

Example

add(int,int)

add(int,int,int)

add(double,double)

All valid.

Name same.

Parameters different.

---

Why?

Because same operation.

Different input.

---

Example

```
static int add(int a,int b)
```

```
{  
    return a+b;  
}
```

```
static int add(int a,int b,int c)
```

```
{  
    return a+b+c;  
}
```

Calls

```
add(5,4);
```

```
add(5,4,6);
```

Java automatically chooses correct method.

---

### **Can Return Type Alone Change?**

Wrong

```
int add(int a,int b)
```

```
double add(int a,int b)
```

Not allowed.

Because parameters same.

Java confused.

---

### **Valid Overloading**

Different number

sum(int,int)

sum(int,int,int)

Different datatype

sum(int,int)

sum(double,double)

Different order

sum(int,double)

sum(double,int)

---

## **Recursion**

Most students fear this.

Actually very easy.

Definition

Method calling itself.

---

Example

Mirror.

Mirror reflects mirror.

Again reflects.

Again reflects.

Recursion also similar.

Method

↓

calls itself

↓

again

↓

again

↓

stops when condition satisfied.

---

### **Why Need Recursion?**

Some problems naturally repeat.

Tree

Folder

Maze

Factorial

Fibonacci

Binary Search

DFS

Merge Sort

Quick Sort

---

### **Base Case**

Most important.

Without base case

Infinite recursion.

Memory full.

Program crash.

Example

hello()

{

hello();

}



Calls forever.

Eventually

StackOverflowError

---

Need

if(condition)

return;

Base case.

---

## **Recursive Flow**

Example

fun(3)

Calls

fun(3)

↓

fun(2)

↓

fun(1)

↓

fun(0)

↓

stop

↓

back

↓

back

↓

back

---

## Factorial

Meaning

5!

=

$5 \times 4 \times 3 \times 2 \times 1$

=

120

Formula

$n!$

=

$n \times (n-1)!$

Recursive Code

```
static int factorial(int n)
{
    if(n==0)
        return 1;

    return n*factorial(n-1);
}
```

---

**Dry Run: factorial(5)**

factorial(5)

=  $5 \times \text{factorial}(4)$

=  $5 \times 4 \times \text{factorial}(3)$

=  $5 \times 4 \times 3 \times \text{factorial}(2)$

=  $5 \times 4 \times 3 \times 2 \times \text{factorial}(1)$

=  $5 \times 4 \times 3 \times 2 \times 1 \times \text{factorial}(0)$

factorial(0)=1

Now return starts

1

↓

$$1 \times 1 = 1$$

↓

$$2 \times 1 = 2$$

↓

$$3 \times 2 = 6$$

↓

$$4 \times 6 = 24$$

↓

$$5 \times 24 = 120$$

Output

120

---

## Fibonacci

Series

0 1 1 2 3 5 8 13 21

Formula

$\text{fib}(n)$

=

$\text{fib}(n-1) + \text{fib}(n-2)$

Base

$\text{fib}(0)=0$

$\text{fib}(1)=1$

Code

```
static int fib(int n)
```

```
{
```

```
    if(n<=1)
```

```
        return n;
```

```
    return fib(n-1)+fib(n-2);
```

```
}
```

---

**Dry Run: fib(5)**

$\text{fib}(5)$

=

$\text{fib}(4)+\text{fib}(3)$

↓

$\text{fib}(3)+\text{fib}(2)+\text{fib}(2)+\text{fib}(1)$

↓

Eventually

5

Tree grows quickly because same values are recalculated many times.

---

## Recursion vs Loop

| Loop                            | Recursion                                                               |
|---------------------------------|-------------------------------------------------------------------------|
| Uses for/while                  | Method calls itself                                                     |
| Faster for many simple problems | Often easier for recursive problems (trees, graphs, divide-and-conquer) |
| Uses less memory                | Uses call stack memory                                                  |
| Easier for beginners            | Takes practice to visualise                                             |

---

## Common Mistakes

### 1. Forgetting to call method

Created

```
hello();
```

If not called

Nothing happens.

---

### 2. Wrong parameters

```
add(5);
```

Needs two inputs.

---

### 3. Wrong return type

```
int hello()
```

```
{  
  
}
```

Must return int.

---

### 4. Returning wrong datatype

```
return "Hello";
```

When return type is int.

Compile Error.

---

## 5. Infinite recursion

No base case.

Program crashes with StackOverflowError.

---

### Program 1: Calculator Using Methods

```
public class Main {
```

```
    static int add(int a, int b) {  
        return a + b;  
    }
```

```
    static int subtract(int a, int b) {  
        return a - b;  
    }
```

```
    static int multiply(int a, int b) {  
        return a * b;  
    }
```

```
    static double divide(int a, int b) {  
        if (b == 0) {  
            System.out.println("Division by zero is not allowed.");  
            return 0;  
        }  
        return (double) a / b;  
    }
```

```
}

public static void main(String[] args) {

    int a = 20;
    int b = 10;

    System.out.println("Addition = " + add(a, b));
    System.out.println("Subtraction = " + subtract(a, b));
    System.out.println("Multiplication = " + multiply(a, b));
    System.out.println("Division = " + divide(a, b));
}
}
```

### **Expected Output**

Addition = 30

Subtraction = 10

Multiplication = 200

Division = 2.0

---

### **Program 2: Recursive Factorial**

```
public class Main {

    static int factorial(int n) {
        if (n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }
}
```



```
public static void main(String[] args) {  
    int number = 5;  
    System.out.println("Factorial = " + factorial(number));  
}  
}
```

### Output

Factorial = 120

---

### Program 3: Recursive Fibonacci

```
public class Main {  
  
    static int fibonacci(int n) {  
        if (n <= 1) {  
            return n;  
        }  
        return fibonacci(n - 1) + fibonacci(n - 2);  
    }  
  
    public static void main(String[] args) {  
  
        int terms = 10;  
  
        for (int i = 0; i < terms; i++) {  
            System.out.print(fibonacci(i) + " ");  
        }  
    }  
}
```

### Output

0 1 1 2 3 5 8 13 21 34

---

## Interview Questions (Must Know)

1. What is a method in Java?
2. Why do we use methods?
3. What is the syntax of a method?
4. What is the difference between void and a non-void return type?
5. What is the purpose of the return statement?
6. What are parameters and arguments?
7. What is the difference between formal parameters and actual arguments?
8. What is method overloading?
9. Can we overload methods by changing only the return type? Why not?
10. What is recursion?
11. Why is a base case required in recursion?
12. What is StackOverflowError in recursion?
13. Explain the recursive flow of factorial.
14. Explain how Fibonacci recursion works.
15. When would you prefer recursion over loops?

---

## Quick Revision Sheet

- **Method** → Reusable block of code.
- **Method Call** → Executes the method.
- **Parameter** → Input defined in the method.
- **Argument** → Actual value passed during the call.
- **Return Type** → Type of value the method sends back.
- **void** → Returns nothing.
- **return** → Ends the method and sends a value back.
- **Method Overloading** → Same method name, different parameter list.
- **Recursion** → A method calling itself.
- **Base Case** → Condition that stops recursion.

- **Factorial Formula**  $\rightarrow n! = n \times (n - 1)!$
- **Fibonacci Formula**  $\rightarrow \text{fib}(n) = \text{fib}(n - 1) + \text{fib}(n - 2)$